PROJECT DOCUMENTATION

SMART TRAFFIC SIGNAL SCHEDULING SYSTEM

Using Stack and Queue

Project Type	Data Structures and Algorithms (DSA) Mini Project
Programming Language	C++
Data Structures	Queue (FIFO) and Stack (LIFO)
Application	Educational traffic-signal scheduling simulation

Prepared as academic documentation. Add student name, roll number, class, and college details before submission.

CONTENTS

- 1. Abstract
- 2. Introduction
- 3. Problem Statement
- 4. Objectives
- 5. Scope
- 6. Hardware and Software Requirements
- 7. Data Structures Used
- 8. System Design and Working
- 9. Algorithm
- 10. Pseudocode
- 11. C++ Implementation Overview
- 12. Sample Input and Output
- 13. Time and Space Complexity
- 14. Advantages
- 15. Limitations and Future Enhancements
- 16. Test Cases
- 17. Conclusion
- 18. References

1. Abstract

This project demonstrates a Smart Traffic Signal Scheduling System in C++ using a queue and a stack. The queue maintains the order of traffic signals and processes them using FIFO (First In, First Out). Once a signal completes its simulated turn, it is reinserted at the rear of the queue to keep the cycle running. A stack stores completed signal activations using LIFO (Last In, First Out), so the most recent signal appears first when viewing history. The project illustrates practical uses of two fundamental data structures.

2. Introduction

Traffic intersections require an orderly sequence for granting movement to different road directions. In this academic simulation, each signal has a name and green-light duration. The queue handles the schedule, while the stack keeps a history of completed activations. The program focuses on data-structure operations rather than real-world traffic control.

3. Problem Statement

Design and implement a system that accepts traffic signal details, places them in a FIFO queue, activates one signal at a time, and repeats the schedule by placing completed signals at the rear. Each completed activation is also pushed onto a LIFO stack. The program should display the queue, show the active signal and its duration, allow the user to stop scheduling, and optionally display signal history.

4. Objectives

- Represent signals using a name and green duration.
- Use a queue for FIFO scheduling.
- Repeat the signal cycle by requeuing completed signals.
- Use a stack to store activation history.
- Display active signal information and signal history.
- Validate basic user input and stop safely.

5. Scope

The project is a console-based academic simulation suitable for demonstrating queue and stack operations. Signal order is based on the order entered by the user. The duration is shown in output, while the user presses Enter to advance to the next signal.

6. Hardware and Software Requirements

- Computer or laptop with keyboard and display.
- Windows, Linux, or macOS.
- C++ compiler supporting C++11 or later (GCC/MinGW or equivalent).
- C++ IDE or editor such as VS Code, Code::Blocks, or Visual Studio.

7. Data Structures Used

Queue (FIFO)

A queue inserts elements at the rear and removes them from the front. The signal at the front receives its turn first. After completing its turn, it is inserted at the rear so the schedule can repeat.

Stack (LIFO)

A stack inserts and removes elements at the top. Each completed signal is pushed onto the history stack. Viewing the stack from top to bottom shows the most recently activated signal first.

8. System Design and Working

- Read the number of signals and each signal's name and duration.
- Insert the signals into the queue in the order entered.
- Display a copy of the queue from front to rear.
- Remove the front signal and display it as GREEN with its duration.
- If the user presses Enter, mark it inactive, push it onto the history stack, and reinsert it at the queue rear.
- If the user enters q, restore the current signal to the front and stop.
- Optionally display the signal history from the top of a stack copy.

Example: Initial queue North → East → South. After North completes its turn, the queue becomes East → South → North, and the history stack contains North at its top. After East completes, the queue becomes South → North → East, and East becomes the top of the history stack.

9. Algorithm

1. Start and initialize empty queue Q and stack H.

2. Read positive signal count n; reject invalid count.

3. Read each signal name and positive duration; insert each signal into Q.

4. Display Q using a temporary queue copy.

5. If the user starts scheduling, repeat: take Q.front(), then Q.pop().

6. Display the selected signal and its green duration.

7. Read user command. If q, restore the current signal to the front and stop.

8. Otherwise push the completed signal onto H and push it to the rear of Q.

9. Repeat until stopped.

10. If requested, display H using a temporary stack copy.

11. End.

10. Pseudocode

```
START
  Create empty queue Q and stack H
  Read n; validate n > 0
  For each signal:
      Read name and positive duration
      Enqueue signal in Q
  Display Q
  WHILE user has not stopped:
      current = Q.front()
      dequeue Q
      Display current as GREEN and its duration
      Read command
      IF command is q:
          restore current to front of Q
          BREAK
      ELSE:
          push current onto H
          enqueue current at rear of Q
  If requested, display a copy of H from top to bottom
END
```

11. C++ Implementation Overview

The source file uses the C++ Standard Library containers std::queue and std::stack. The Signal structure stores a signal name and green duration. TrafficSignalSystem manages the queue and stack and provides functions to add signals, display the queue, run scheduling, and display history. The scheduling loop is interactive: Enter advances after a simulated interval, and q stops the process. The duration is displayed but not automatically timed.

12. Sample Input and Output

Sample input

```
Number of signals: 3
North Road — 30 seconds
East Road — 25 seconds
South Road — 20 seconds
```

Sample output

```
Signal Queue (front to rear):
North Road | 30 seconds
East Road | 25 seconds
South Road | 20 seconds

Active Signal: North Road
Status: GREEN
Green duration: 30 seconds
North Road signal is now RED.
```

The next queue order becomes East Road → South Road → North Road. The stack history shows North Road first, because it is the most recent completed activation.

13. Time and Space Complexity

- Initial queue creation: O(n) total, assuming each enqueue is O(1).
- Displaying the queue: O(n).
- Removing and re-inserting one signal: O(1) per turn.
- Pushing one history entry: O(1).

- Space complexity: O(n) for the queue plus O(k) history entries after k completed turns.

14. Advantages

- Queue provides a simple FIFO schedule.
- Stack shows recent signal activations in LIFO order.
- The schedule can repeat without rebuilding the queue.
- The program demonstrates two common data structures in one application.

15. Limitations and Future Enhancements

Limitations

- Educational simulation only; it does not control actual traffic lights.
- The duration is displayed but is not automatically timed.
- The schedule is fixed in the order entered and does not use live traffic data.

Future enhancements

- Add automatic countdown and timed signal changes.
- Allow adding, deleting, or editing signals through a menu.
- Simulate traffic density to vary timing in a controlled demo.
- Add a graphical interface with signal colors and a countdown display.

16. Test Cases

Test	Condition	Expected Result
1	Three valid signals	Signals appear in entered FIFO order.
2	One completed turn	Signal moves from queue front to rear and is pushed onto stack.
3	Enter q	Scheduling stops and current signal is restored to queue front.
4	Zero/negative signal count	Program reports invalid input and exits.
5	Zero/negative duration	Program reports invalid input and exits.
6	View history	Most recently completed signal appears first.

17. Conclusion

The Smart Traffic Signal Scheduling System shows how a queue and a stack can work together in a simple scheduling application. The queue maintains the repeating FIFO order of signals, while the stack records completed activations in LIFO order. The project strengthens understanding of enqueue, dequeue, push, stack traversal, queue traversal, and basic input validation.

18. References

- C++ Standard Library reference: https://en.cppreference.com/
- Data Structures and Algorithms course notes on stacks and queues.
- Project algorithm and C++ source code prepared for this academic mini project.